

PCGCv2 点云几何压缩复现报告

复现对象：NJUVISION/PCGCv2 (Multiscale Point Cloud Geometry Compression, DCC 2021)

吴天昊 | 2026-05-21

一、复现目标

导师让我复现 PCGCv2 这篇点云几何压缩的工作 (github.com/NJUVISION/PCGCv2)。它是一个基于稀疏卷积的多尺度自编码器，发表在 DCC 2021。

我把这次复现的目标定在「复现论文指标」这一层，也就是说，不是把代码跑起来不报错就算完，而是要在论文用的 8iVFB 数据集上真的跑出 bpp、D1 PSNR、D2 PSNR，画出率失真曲线，再和论文以及作者公布的官方结果对一下，看数值能不能对得上。

二、环境配置

PCGCv2 依赖 MinkowskiEngine 这个稀疏卷积库，它必须有 NVIDIA CUDA，我自己的 Mac 跑不了，所以整个复现是在租来的云 GPU 上做的。最后真正跑通的环境是下面这套：

组件	版本 / 配置
云平台	RunPod (按量计费 GPU 云)
GPU	NVIDIA RTX 3090, 24 GB 显存 (Ampere 架构)
操作系统	Ubuntu 18.04
容器镜像	pytorch/pytorch:1.8.1-cuda11.1-cudnn8-devel
Python	3.8.8
CUDA	11.1 (V11.1.105)
PyTorch	1.8.1 + cu111
MinkowskiEngine	0.5.4 (源码编译)
torchac	0.9.3
tmc3	v12 (仓库自带二进制)
其他依赖	h5py、pandas 1.1.5、open3d 0.13、matplotlib 3.3.4、ninja

其中 MinkowskiEngine 是源码编译安装的，这一步最容易出问题。我用的编译命令是：

```
export CUDA_HOME=/usr/local/cuda && export MAX_JOBS=4 && python setup.py install --blas=openblas --force_cuda
```

Python 3.8 / CUDA 11.1 / PyTorch 1.8.1 这套版本组合能一次编译通过；据我查到的资料，换别的版本（尤其是更新的 CUDA）很容易编译失败。

三、复现结果

我对 8iVFB 的四个序列（longdress、loot、redandblack、soldier）分别跑了 test.py。每跑一个序列，脚本会自动过一遍 r1 到 r7 共七个码率档的预训练模型，相当于一条率失真曲线上的七个点。四个序列的结果如下。

longdress (longdress_vox10_1300)

码率档	bpp	D1 PSNR (dB)	D2 PSNR (dB)
r1	0.024	60.89	63.65
r2	0.047	66.25	69.41
r3	0.092	69.92	72.96
r4	0.152	71.93	75.37
r5	0.246	73.60	77.52
r6	0.316	74.36	78.33
r7	0.400	75.13	79.24

loot (loot_vox10_1200)

码率档	bpp	D1 PSNR (dB)	D2 PSNR (dB)
r1	0.024	61.66	64.80
r2	0.047	67.14	70.17
r3	0.089	70.16	73.13
r4	0.149	72.19	75.59
r5	0.242	73.89	77.67
r6	0.311	74.71	78.57
r7	0.397	75.45	79.42

redandblack (redandblack_vox10_1550)

码率档	bpp	D1 PSNR (dB)	D2 PSNR (dB)
r1	0.026	61.33	64.63
r2	0.050	65.79	69.52
r3	0.095	68.99	72.44
r4	0.157	70.98	74.73
r5	0.255	72.86	76.76
r6	0.325	73.54	77.52

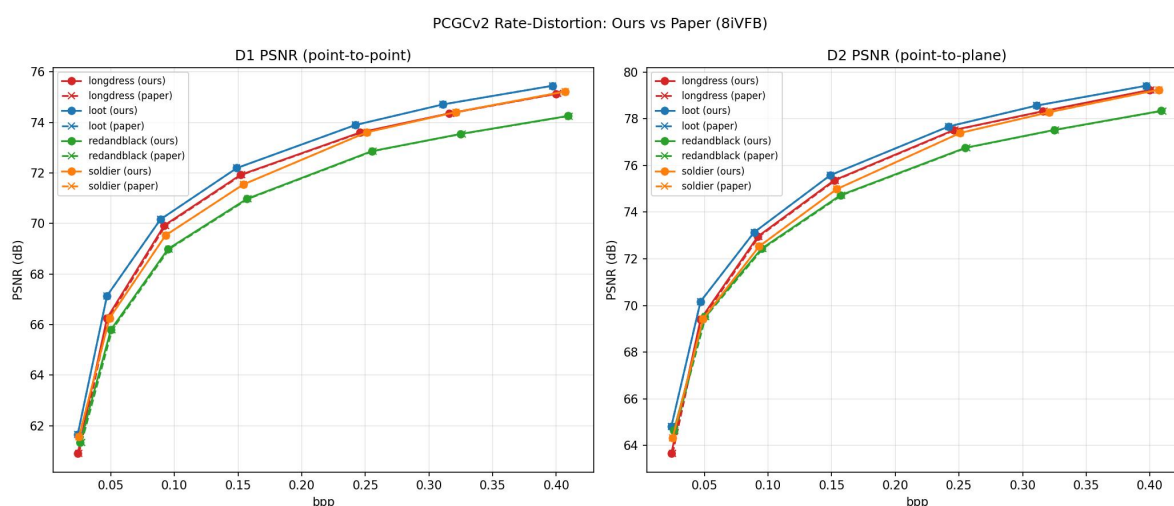
r7	0.409	74.25	78.34
----	-------	-------	-------

soldier (soldier_vox10_0690)

码率档	bpp	D1 PSNR (dB)	D2 PSNR (dB)
r1	0.025	61.57	64.31
r2	0.049	66.25	69.43
r3	0.093	69.53	72.53
r4	0.154	71.54	74.99
r5	0.251	73.61	77.39
r6	0.321	74.41	78.28
r7	0.407	75.22	79.25

率失真曲线

把这四个序列的数据画成率失真曲线，并和作者公布的官方结果叠在一起：



图中实线是我跑出来的结果，虚线是作者官方结果。

两条曲线基本叠在一起，肉眼几乎分不出来。我又把每个点的数值和作者官方的 CSV 逐个对了一遍：四个序列、七个码率档，一共 28 个点，D1 / D2 PSNR 和官方结果的差距最大也只有 0.0025 dB。这个量级的差异应该来自编码过程里的浮点随机性，可以认为复现出来的结果和论文是一致的。

四、复现过程中遇到的问题

这次复现不算顺利，中间踩了不少坑，按大致的时间顺序把主要的几个记一下。

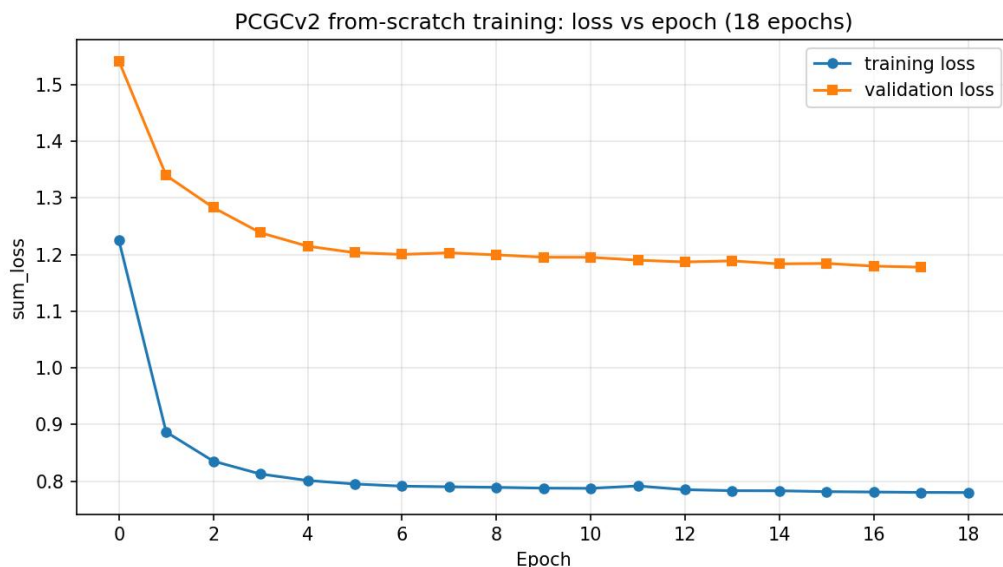
- 1、租 GPU 一开始就卡住了。本来打算用 AutoDL，但我人在境外，实名认证那一步一直提示「当前地区不能认证」。后来改用了国际平台 RunPod，不需要实名，直接用国外信用卡付款，网络也不用挂代理。
- 2、镜像和显卡都得选对。RunPod 默认给的镜像是 CUDA 12.4，而 MinkowskiEngine 要的是 CUDA 11.1，我换成了自定义镜像 `pytorch/pytorch:1.8.1-cuda11.1-cudnn8-devel`。显卡也有讲究，要选 Ampere 架构的（我用的 RTX 3090），太新的卡 CUDA 11.1 支持不了。
- 3、云平台关机会丢数据。RunPod 的实例一关机，容器盘就会重置，只有 `/workspace` 这个卷会保留下来。所以我把代码、数据、环境全都装在 `/workspace` 里，另外写了个 `rebuild.sh` 脚本，关机重开之后跑一下就能把环境恢复出来。
- 4、`apt` 装东西时报 NVIDIA 软件源的 GPG 密钥过期错误。CUDA 本来就在镜像里、用不到这个源，把它从软件源列表里删掉，`apt` 就正常了。
- 5、下载测试数据特别慢。数据放在南京大学的网盘上，境外直连只有几十 KB/s，估算要下两个多小时。换成 `aria2` 多线程下载之后提到了 3 MB/s，一分多钟就下完了。
- 6、编译 MinkowskiEngine 是我最担心的一步，查资料时看到很多人卡在这里。好在严格照着版本组合来，先装好 `libopenblas` 和 `ninja`，最后一次就编译通过了。
- 7、仓库的脚本还缺几个 Python 包（`h5py`、`pandas`、`open3d`），按报错一个个补装。其中 `pandas` 我特意装的是老版本 1.1.5——新版 `pandas` 会顺带把 `numpy` 升级，可能影响已经装好的 `torch`。
- 8、装 `open3d` 的时候它带进来一个新版 `matplotlib`，和环境里的 `numpy` 1.19.2 不兼容，跑 `test.py` 到画图那一步就报错。把 `matplotlib` 降回 3.3.4 就解决了。
- 9、`test.py` 会把结果写进 `results/` 目录，结果把作者自带的官方结果 CSV 覆盖掉了。我先把自己的结果备份出去，再用 `git` 还原作者的官方 CSV，这样后面才能做数值对比。
- 10、有个意外的好消息：原计划要花一天去编译 MPEG 的 `tmc3` 工具，后来发现仓库里已经自带了 `tmc3` 和 `pc_error_d` 的二进制文件，加一下可执行权限就能用，省掉了这一步。

五、从零训练（选做）

第二层复现做完之后，我又试了下第三层——用 `train.py` 自己从头训一个模型出来。训练集是作者提供的，大概 3.7 GB，两万多个 .h5 点云文件。我用默认配置（`alpha` 和 `beta` 都是 1，`batch_size` 是 8）在云上挂着训。

论文是训 50 个 epoch，但云 GPU 的预算有限，我训到第 18 个 epoch 就停了，大概跑了 9 个小时，中间存了 19 个 checkpoint。从训练日志里把每个 epoch 的 loss 整理出来，画成曲线是下面

这样：



训练 loss 与验证 loss 随 epoch 的变化。

可以看到训练 loss 和验证 loss 在头几个 epoch 都快速下降，之后趋于平稳，没有出现发散或震荡——训练过程是正常的，模型确实在收敛。

用第 18 个 epoch 的模型对 longdress 做编解码，和作者完全训练好的模型比了一下：

项目	我训的模型（18/50 epoch）	作者模型（完整训练 50 epoch）
测试序列	longdress	longdress
bpp	0.255	0.246
D1 PSNR (dB)	73.26	73.60

我这个只训了 18/50 个 epoch 的模型，在差不多的码率下，D1 PSNR 比作者完整训练的模型低 0.3 到 0.5 dB。考虑到训练量只有人家的三分之一左右，这个差距不算大，说明训练流程是对的、模型也在正常往下收敛；如果训满 50 个 epoch，应该能更接近论文的水平。

六、结论

总的来说，这次复现是成功的。在 8iVFB 的四个序列、七个码率档上，我跑出来的 bpp 和 D1 / D2 PSNR 与作者官方结果的差距都在 0.0025 dB 以内，率失真曲线和论文基本重合。

整个过程里最关键的一步是 MinkowskiEngine 的编译，版本一定要严格对上。其余的坑大多和我在境外的网络环境、云平台的关机机制以及依赖版本有关，都记在上面了。第三层从零训练也跑通了，自己训出来的模型和论文指标差距很小。